FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO — FECAP

CIÊNCIA DA COMPUTAÇÃO — 6º SEMESTRE

SMART TENNIS ARENA

ENTREGA 2 — TEORIA DA COMPUTAÇÃO E LINGUAGENS FORMAIS

Algoritmos de otimização e análise de complexidade

Integrantes:

Flavia da Costa R. Faria – 20021548

Guilherme Muniz Gomes – 24026572

Lucas Moreira de Godoy 24026298

Maria Eduarda Cabral Foloni - 24026502

São Paulo — SP
2026

1 Contexto e objetivo

O Score Smash é uma estrutura de jogo para treino de pontaria em que a bola é detectada por feixes de laser e sensores LDR, dispensando a conferência manual dos acertos. O jogo faz parte da proposta da Smart Tennis Arena e reúne sistemas embarcados, IoT, banco de dados e desenvolvimento de software em um único produto.

A área de jogo é organizada em linhas e colunas. Quando a bola corta um feixe, o Arduino Uno lê o sensor correspondente, com apoio dos multiplexadores 74HC4067, e obtém a coordenada (X, Y) do acerto. A coordenada é convertida em pontos, os LEDs mostram o resultado e os dados seguem por uma API para o banco de dados e o dashboard.

Para que esse conjunto se comporte de modo previsível, é preciso dizer em que momentos cada ação é permitida e o que acontece quando algo dá errado. Este relatório faz isso com a Teoria da Computação: descreve o sistema como um autômato, fixa as regras que o governam, declara o que deve valer antes e depois de cada operação, justifica o modelo escolhido e discute o que é possível decidir sobre ele.

2 Alfabeto de eventos

O sistema reage a treze eventos, que formam o alfabeto do modelo:

Σ = {lig, des, vok, ini, ace, prc, sin, pau, ret, fin, env, err, rei}

Cada símbolo tem um significado fixo, mas só produz efeito nos estados em que existe uma transição para ele.

Símbolo	Evento	Significado	Estados em que é aceito
lig	Ligar	Energização do sistema	E0
des	Desligar	Pedido de desligamento	E1, E2, E6, E7, E8
vok	Verificação concluída	Sensores LDR e lasers respondendo corretamente	E1
ini	Iniciar partida	Comando que abre a partida	E2
ace	Acerto detectado	Feixe interrompido em alguma região da matriz	E3
prc	Acerto processado	Coordenada (X, Y) obtida e pontuação atualizada	E4

Símbolo	Evento	Significado	Estados em que é aceito
sin	Sinalização concluída	LEDs terminaram de indicar o resultado	E5
pau	Pausar	Pedido de pausa da partida	E3
ret	Retomar	Pedido para continuar a partida pausada	E6
fin	Finalizar	Comando que encerra a partida	E3
env	Envio confirmado	A API confirmou o recebimento dos resultados	E7
err	Falha	Erro de leitura, de hardware ou de comunicação	E1 a E7
rei	Reiniciar	Pedido de reinicialização após uma falha	E8

3 Autômato do Score Smash

Definição formal

O comportamento operacional é descrito pelo Autômato Finito Determinístico M = (E, Σ, δ, E0, F), no qual:

E = {E0, E1, E2, E3, E4, E5, E6, E7, E8} é o conjunto de estados;

Σ é o alfabeto da seção 2;

δ : E × Σ → E é a função de transição, parcial, listada na seção 3.3;

E0 é o estado inicial;

F = {E0} é o conjunto de estados de aceitação, pois uma execução bem-sucedida termina com o sistema desligado.

Estados

- **E0 - Desligado:** sistema em repouso.
- **E1- Inicializando e verificando sensores:** o Arduino, os multiplexadores e os LEDs são iniciados e a leitura de cada sensor LDR é conferida com os lasers ligados.
- **E2 - Pronto:** verificação feita; o sistema espera o comando de início.
- **E3 - Monitoramento:** partida em andamento, com leitura contínua dos sensores à espera de um feixe interrompido.
- **E4 - Identificação do acerto:** o sistema descobre a coordenada (X, Y) atingida e aplica a regra de pontuação.
- **E5 - Sinalização:** os LEDs mostram a região atingida e o resultado.
- **E6 - Pausa:** a partida fica suspensa e nenhum acerto é contado.
- **E7 - Encerramento e envio:** a pontuação é consolidada e enviada à API.
- **E8 - Falha:** estado de exceção, do qual só se sai por reinício ou desligamento.

Transições

A tabela lista, para cada estado, os eventos que ele aceita e o destino de cada um. Qualquer par estado-evento que não apareça aqui não tem transição definida.

Estado	Transições definidas
E0 Desligado	lig → E1
E1 Inicialização	vok → E2; des → E0; err → E8
E2 Pronto	ini → E3; des → E0; err → E8
E3 Monitoramento	ace → E4; pau → E6; fin → E7; err → E8
E4 Identificação do acerto	prc → E5; err → E8
E5 Sinalização	sin → E3; err → E8
E6 Pausa	ret → E3; des → E0; err → E8
E7 Encerramento e envio	env → E0; des → E0; err → E8
E8 Falha	rei → E1; des → E0

São 23 transições no total. Em uma partida sem incidentes, o sistema percorre o ciclo nominal

lig vok ini (ace prc sin)* fin env, em que o trecho entre parênteses se repete a cada acerto. Os desvios são a pausa (pau e ret), o desligamento antecipado (des) e a falha (err), seguida de rei ou des.

A Figura 1, com o desenho do autômato, pode ser montada diretamente a partir da tabela acima.

4 Regras de funcionamento

As regras abaixo estão agrupadas pela fase da operação a que se aplicam.

Preparação

1. O sistema só sai do repouso pelo evento lig, que leva a E1.
2. A partida só pode começar depois de vok, isto é, depois que todos os sensores e lasers foram verificados.
3. O evento ini só é aceito em E2. Antes disso não existe partida.

Detecção e pontuação

4. A leitura dos sensores por meio dos multiplexadores acontece em E3, o único estado em que um acerto pode ser registrado.
5. Ao detectar um feixe interrompido (ace), o sistema vai a E4 e identifica a coordenada (X, Y).

6. Em E4, a coordenada é associada à pontuação definida para a região e a pontuação do jogador é atualizada. Só então prc leva a E5.
7. Em E5, os LEDs indicam o resultado. Terminada a sinalização (sin), o sistema volta a E3 e está pronto para o próximo acerto.

Interrupções e encerramento

8. O evento pau leva de E3 a E6. Em E6, ace não tem transição, de modo que nenhum acerto é contado durante a pausa; ret devolve o sistema a E3.
9. O evento fin leva de E3 a E7, onde os resultados são consolidados e enviados à API.
10. O evento env, que confirma o recebimento, leva de E7 a E0. O desligamento por des também é aceito em E7.
11. O desligamento direto (des) é aceito em E1, E2, E6, E7 e E8. Em E3, E4 e E5 ele não existe: a partida precisa ser pausada ou finalizada antes.

Exceções

12. O evento err é aceito de E1 a E7 e leva a E8. A partir daí o fluxo normal fica bloqueado.
13. Em E8, rei reinicia o processo em E1 e des desliga o sistema.
14. Qualquer evento sem transição definida para o estado atual é inválido no modelo. O que o equipamento físico faz com esses eventos (por exemplo, um segundo feixe interrompido enquanto o primeiro acerto ainda é identificado em E4) não está definido aqui e deve ser confirmado com a equipe.

5 Contratos das operações

Cada operação é descrita pelo que precisa ser verdadeiro antes dela e pelo que passa a valer depois.

Ligar o sistema (lig)

- Antes: sistema em E0.
- Depois: sistema em E1, com a inicialização em curso.

Concluir a verificação (vok)

- Antes: sistema em E1 e cada sensor LDR respondendo à incidência do seu laser.
- Depois: sistema em E2, aguardando o início.

Iniciar a partida (ini)

- Antes: sistema em E2, com a verificação concluída.
- Depois: sistema em E3, monitorando a área de jogo.

Registrar um acerto (ace, prc, sin)

- Antes: sistema em E3 e variação de luminosidade em um sensor.
- Depois: coordenada (X, Y) identificada, pontuação do jogador atualizada, resultado indicado nos LEDs e sistema de volta a E3.

Pausar e retomar (pau, ret)

- Antes: E3 para pausar; E6 para retomar.
- Depois: pau leva a E6, sem contagem de acertos; ret devolve o sistema a E3.

Finalizar a partida (fin)

- Antes: sistema em E3 e pedido de encerramento recebido.
- Depois: sistema em E7, consolidando e enviando os resultados.

Concluir o envio (env)

- Antes: sistema em E7 e recebimento confirmado pela API.
- Depois: sistema em E0, encerrado com segurança.

Tratar uma falha (err)

- Antes: sistema em um estado de E1 a E7.
- Depois: sistema em E8, sem prosseguir no fluxo normal.

Recuperar-se de uma falha (rei)

- Antes: sistema em E8 e reinício autorizado.
- Depois: sistema em E1, repetindo a inicialização.

Desligar (des)

- Antes: sistema em E1, E2, E6, E7 ou E8.
- Depois: sistema em E0.

As condições que dependem de leitura de sensores, de confirmação da API e do critério de acerto são interpretações operacionais. Para torná-las verificáveis, a especificação técnica ainda precisa fixar os valores de pontuação de cada região e o limiar de luminosidade que caracteriza um feixe interrompido.

6 Por que um autômato finito determinístico

O modelo escolhido é o AFD, porque o controle do Score Smash tem poucos estados bem separados e a reação a cada evento depende só do estado atual. A tabela compara as três alternativas consideradas.

Modelo	Capacidade	Adequação ao Score Smash
Autômato finito determinístico	Memória finita, um único destino por par estado-evento	Adequado: cobre todo o fluxo de controle com nove estados
Autômato com pilha	Acrescenta uma pilha para estruturas aninhadas	Desnecessário: nenhum comportamento exige empilhar e desempilhar contextos
Máquina de Turing	Memória ilimitada, poder computacional máximo	Excessivo: o fluxo de controle não precisa dessa capacidade

O sistema real guarda a pontuação do jogador e converte coordenadas em pontos, operações que um AFD sozinho não representa. Por isso, o modelo trata a pontuação como um dado

ligado às transições de E4 e E5, e não como parte dos estados. O autômato descreve a ordem permitida das ações, e o cálculo numérico fica fora dele.

O AFD é determinístico porque cada par estado-evento definido leva a no máximo um destino, o que torna a execução previsível e fácil de testar.

7 O que é possível decidir sobre o modelo

Um problema é decidível quando existe um procedimento que sempre termina e responde sim ou não. Como o autômato tem 9 estados e 23 transições, todas as perguntas abaixo são decidíveis.

1. **Um evento é permitido agora?** Basta consultar a tabela de transições. Por exemplo, ini só é aceito em E2 e ace só em E3.
2. **Uma sequência de eventos é aceita?** Parte-se de E0 e aplica-se cada evento em ordem. Se algum não tiver transição, a sequência é rejeitada. Se todos tiverem, ela é aceita quando termina em E0. Uma sequência de n eventos exige no máximo n consultas, logo o procedimento sempre termina.
3. **Um estado é alcançável?** Percorrendo o grafo de transições a partir de E0, vê-se que os nove estados são alcançáveis. Por exemplo, E5 é alcançado por lig vok ini ace prc.
4. **O sistema pode travar?** Em todos os estados existe um caminho de volta a E0, por exemplo fin env a partir de E3 ou des a partir de E8. Não há estado sem saída.
5. **Um acerto pode ser contado durante a pausa?** Não. A única transição por ace parte de E3, e E6 não oferece ace. A propriedade vale para qualquer execução, porque é verificada sobre o grafo inteiro.

Alguns exemplos de sequências:

- *lig vok ini ace prc sin fin env* é aceita: uma partida completa com um acerto, terminada em E0.
- *lig vok ini err rei vok ini fin env* é aceita: a partida sofre uma falha, é reiniciada e termina normalmente.
- *lig des* é aceita: o sistema é desligado ainda durante a inicialização.
- *lig ini* é rejeitada, pois ini não tem transição em E1.
- *lig vok ini ace ace* é rejeitada, pois ace não tem transição em E4.
- *lig vok ini* é processada por inteiro, mas não aceita, pois termina em E3, que não pertence a F.

Essas conclusões valem para o modelo abstrato. Elas não garantem que lasers, sensores, multiplexadores, a rede ou a API estejam sempre funcionando. Esses fatores dependem de condições físicas e de implementação que o autômato não representa.

8 Conclusão

A especificação descreve o Score Smash como um autômato de nove estados e treze eventos, com regras que determinam quando cada ação é permitida, o que acontece em caso de falha e como a partida termina. Os contratos de operação deixam claro o que se espera antes e depois de cada passo, e a análise mostra que as perguntas essenciais sobre o comportamento podem ser respondidas de forma mecânica.

O modelo, porém, trata apenas do fluxo de controle. Precisão dos sensores, alinhamento dos lasers, cálculo da pontuação e transmissão dos dados dependem de elementos que ficam fora do autômato. Ele serve, portanto, como base para conferir a implementação e localizar diferenças entre o comportamento previsto e o observado.